There are several advantages of using UVM over SV:-

a) Reusability, scalability, interoperability all these things can be achieved through methodology only like UVM.

b) plug and play feature of VIP can be achieved through UVM.

c) UVM is a BCL (base class library) which helps you to use certain features of UVM,not possible through SV.

d) The overriding constraints,factory registration can be done through create and utils macro while in SV you will generally use new construct which will be hard coded to a specific type.But UVM create method is not hard coded.

e) SV is quite vast and exhaustive so many switch over to UVM and can be used a common platform so portability and reusability can be done quite easily.

These are few advantages of UVM over SV but as you keep moving with UVM you will find that many functionality and construct are easily done with UVM as compared to SV.

A few different words to the Dave's link:
(1) Modularity: - seperating test form testbench
- using an agent-based approach
(2) Scalability: - usage of parameters,
- instantiation of thge same agent several times
(3) reusabality: - plug-and-play reuse
- reusing components without modifications in another project (providing only a link to the existing agent)
(4) Configurability: - UVM provides different mechanism for configuration
- factory
- uvm_config_db
- configuration objects
(5) Interoperability: Because of the UVM base-class library and the OOP mechanism your components are compatible to each other of the same type.

1. **Question 1. What Is Uvm? What Is The Advantage Of Uvm?**
   Answer :
   UVM (Universal Verification Methodology) is a standardized methodology for verifying the both complex & simple digital design in simple way.
   UVM Features:
   - First methodology & second collection of class libraries for Automation
   - Reusability through test bench
   - Plug & Play of verification IPs
   - Generic Test bench Development
   - Vendor & Simulator Independent
   - Smart Test bench i.e. generate legal stimulus as from pre-planned coverage plan
   - Support CDV –Coverage Driven Verification
   - Support CRV –Constraint Random Verification
   - UVM standardized under the Accelerate System Initiative
   - Register modeling
2. **Question 2. Uvm Derived From Which Language?**
   Answer :
   Here is the detailed connection between SV, UVM, OVM and other methodologies.

3. **Question 3. What Is The Difference Between Uvm_component And Uvm_object? Or We Already Have Uvm_object, Why Do We Need Uvm_component Which Is Actually Derived Class Of Uvm_object?**
   Answer :
   uvm_component:
   - Quasi Static Entity (after build phase it is available throughout the simulation).
   - Always tied to a given hardware (DUT Interface) or a TLM port.
   - Having phasing mechanism for control the behavior of simulation.
   - Configuration Component Topology.

   uvm_object:
   - Dynamic Entity (creates when needed, transfer from one component to other & then dereference).
   - Not tied to a given hardware or any TLM port.
   - Not phasing mechanism.
4. **Question 4. Which Uvm Phase Is Top - Down, Bottom – Up & Parallel?**
   Answer :
   Only build phase is a top-down & other phases are bottom-up except run phase which is parallel. The build phase works top-down since the test bench hierarchy may be configure so we need to build the branches before leafs.
5. **Question 5. Why Build Phase Is Top – Down & Connect Phase Is Bottom – Up?**
   Answer :
   The connect phase is intended to be used for making TLM connections between components, which is why it occur after build phase. It work bottom-up so that its got the correct implementation all the way up the design hierarchy, if worked top-down this would be not possible.
6. **Question 6. Which Phase Is Function & Which Phase Is Task?**
   Answer :
   Only run phase is a task (time consuming phase) & other phases are functions (non-blocking).
7. **Question 7. Which Phase Takes More Time And Why?**
   Answer :
   As previously said the run phase is implemented as task and remaining all are function. run phase will get executed from start of simulation to till the end of simulation. run phase is time consuming, where the test case is running.
8. **Question 8. How Uvm Phases Initiate?**
   Answer :
   UVM phases initiate by calling run test (“test1”) in top module. When run test() method call, it first create the object of test top & then call all phases.
9. **Question 9. How Test Cases Run From Simulation Command Line?**
   Answer :
   In top module write run test(); i.e. Don't give anything in argument.

   Then in command line: +UVM_TESTNAME=test name.
10. **Question 10. Difference Between Module & Class Based Tb?**
    Answer :
    - A module is a static object present always during of the simulation.
    - A Class is a dynamic object because they can come and go during the life time of simulation.
11. **Question 11. What Is Uvm_config_db? What Is Difference Between Uvm_config_db & Uvm_resource_db?**

Answer :

- Uvm_config_db is a parameterized class used for configuration of different type of parameter into the uvm database, So that it can be used by any component in the lower level of hierarchy.
- Uvm_config_db is a convenience layer built on top of uvm_resource_db, but that convenience is very important. In particular, uvm_resource_db uses a "last write wins" approach. The uvm_config_db, on the other hand, looks at where things are in the hierarchy up through end_of_elaboration, so "parent wins." Once you start start_of_simulation, the config_db becomes "last write wins."
- All of the functions in uvm_config_db#(T) are static, so they must be called using the :: operator. It is extended from the uvm_resource_db#(T), so it is child class of uvm_resource_db#(T).

12. **Question 12. What Is The Advantage And Difference Of `uvm_component_utils() And `uvm_object_utils()?**

Answer :

- The utile macros define the infrastructure needed to enable the object/component for correct factory operation.
- The reason there are two macros is because the factory design pattern fixes the number of arguments that a constructor can have. Classes derived from uvm_object have constructors with one argument, a string name. Classes derived from uvm_component have two arguments, a name and a uvm_component parent.
- The two `uvm_*utile macros inserts code that gives you a factory create() method that delegates calls to the constructors of uvm_object or uvm_component. You need to use the respective macro so that the correct constructor arguments get passed through. This means that you cannot add extra constructor arguments when you extend these classes in order to be able to use the UVM factory.

13. **Question 13. Difference Between `uvm_do And `uvm_rand_send ?**

Answer :

uvm_do perform the below steps:

- Create
- Start item
- Randomize
- Finish item
- get response (optional)

While `uvm_rand_send perform all the above steps except create. User needs to create sequence / sequence item.

Question 14. Difference Between Uvm_transaction And Uvm_seq_item?

Answer :

class uvm_sequence_item extends uvm_transaction

Uvm_sequence_item extended from uvm_transaction only, uvm_sequence_item class has more functionality to support sequence & sequencer features. Uvm_sequence_item provides the hooks for sequencer and sequence, So you can generate transaction by using sequence and sequencer , and uvm_transaction provide only basic methods like do print and do record etc .

Question 15. Is Uvm Is Independent Of Systemverilog?

Answer :

UVM is a methodology based on Systemverilog language and is not a language on its own. It is a standardized methodology that defines several best practices in verification to enable efficiency in terms of reuse and is also currently part of IEEE 1800.2 working group.

Question 16. What Are The Benefits Of Using Uvm?

Answer :

Some of the benefits of using UVM are:

- **Modularity and Reusability** – The methodology is designed as modular components (Driver, Sequencer, Agents , env etc) which enables reusing components across unit level to multi-unit or chip level verification as well as across projects.
- **Separating Tests from Test benches** – Tests in terms of stimulus/sequencers are kept separate from the actual test bench hierarchy and hence there can be reuse of stimulus across different units or across projects.
- **Simulator independent** – The base class library and the methodology is supported by all simulators and hence there is no dependence on any specific simulator.
- **Better control on Stimulus generation** – Sequence methodology gives good control on stimulus generation. There are several ways in which sequences can be developed which includes randomization, layered sequences, virtual sequences etc which provides a good control and rich stimulus generation capability.
- **Easy configuration** – Config mechanisms simplify configuration of objects with deep hierarchy. The configuration mechanism helps in easily configuring different test bench components based on which verification environment uses it and without worrying about how deep any component is in test bench hierarchy.
- **Factory mechanism** – Factory mechanisms simplifies modification of components easily. Creating each components using factory enables them to be overridden in different tests or environments without changing underlying code base.

Question 17. Can We Have User Defined Phase In Uvm?

Answer :

In addition to the predefined phases available in uvm , the user has the option to add his own phase to a component. This is typically done by extending the uvm_phase class the constructor needs to call super. new which has three arguments.

- Name of the phase task or function
- Top down or bottom up phase
- Task or function

The call task or call_func and get_type_name need to be implemented to complete the addition of new phase.

Below is a simple example

Example:

```
Class custom phase extends uvm_phase;
  Function new ();
    Super. New (“custom”, 1, 1);
  End function
 Task call task (uvm_component parent);
   My_comp_type comp;
```

```
  If ( $cast(comp, parent) )
        comp.custom phase ();
 End task
 Virtual function string get_type_name ();
   Return “custom”;
 End function
End class
```

Question 18. What Is Uvm Ral Model? Why It Is Required?

Answer :

In a verification context, a register model (or register abstraction layer) is a set of classes that model the memory mapped behavior of registers and memories in the DUST in order to facilitate stimulus generation and functional checking (and optionally some aspects of functional coverage). The UVM provides a set of base classes that can be extended to implement comprehensive register modeling capabilities.

Question 19. What Is The Difference Between New() And Create?

Answer :

We all know about new () method that is use to allocate memory to an object instance. In UVM (and OVM), the create () method causes an object instance to be created from the factory. This allows you to use factory overrides to replace the desired object with an object of a different type without having to recode.

The advantage of create() over new() is that when create() method is called, it will check if there is a factory override and constructs the object of override type.

Question 20. What Is Analysis Port?

Answer :

Analysis port (class uvm_tlm_analysis_port) — a specific type of transaction-level port that can be connected to zero, one, or many analysis exports and through which a component may call the method write implemented in another component, specifically a subscriber.

port, export, and imp classes used for transaction analysis.

uvm_analysis_port
Broadcasts a value to all subscribers implementing a uvm_analysis_imp.

uvm_analysis_imp
Receives all transactions broadcasted by a uvm_analysis_port.

uvm_analysis_export
Exports a lower-level uvm_analysis_imp to its parent.

Question 21. What Is Tlm Fifo?

Answer :

In simpler words TLM FIFO is a FIFO between two UVM components, preferably between Monitor and Scoreboard. Monitor keep on sending the DATA, which will be stored in TLM FIFO, and Scoreboard can get data from TLM FIFO whenever needed.

```
// create a FIFO with depth 4
  tlm_fifo = new ("uvm tlm_fifo", this, 4);
```

Question 22. How Sequence Starts?

Answer :

Start item starts the sequence

Virtual task start item (uvm_sequence_item item,

int set priority = -1,

Uvm_sequencer_base sequencer = null)

Start item and finish item together will initiate operation of a sequence item. If the item has not already been initialized using create item, then it will be initialized here to use the default sequencer specified by m_sequencer.

Question 23. What Is The Difference Between Uvm Ral Model Backdoor Write/read And Front Door Write/read?

Answer :

- Font door access means using the standard access mechanism external to the DUTY to read or write to a register. This usually involves sequences of time-consuming transactions on a bus interface.
- Backdoor access means accessing a register directly via hierarchical reference or outside the language via the PLI. A backdoor reference usually in 0 simulation time.

Question 24. What Is Objection?

Answer :

- The objection mechanism in UVM is to allow hierarchical status communication among components which is helpful in deciding the end of test.
- There is a built-in objection for each in-built phase, which provides a way for components and objects to synchronize their testing activity and indicate when it is safe to end the phase and, ultimately, the test end.
- The component or sequence will raise a phase objection at the beginning of an activity that must be completed before the phase stops, so the objection will be dropped at the end of that activity. Once all of the raised objections are dropped, the phase terminates.

Raising an objection: phase.raise_objection (this);
Dropping an objection: phase.drop_objection (this);

Question 25. What Is M_sequencer? Or Difference Between M_sequencer And M_sequencer?

Answer :

M_sequencer is the default handle for uvm_vitual_sequencer and m_sequencer is the hook up for child sequencer.

M_sequencer is the generic uvm_sequencer pointer. It will always exist for the uvm_sequencer and is initialized when the sequence is started.

P_sequencer is a typed-specific sequencer pointer, created by registering the sequence to the sequencer using macros (`uvm_declare_p_sequencer) . Being type specific, you will be able to access anything added to the sequencer (i.e. pointers to other sequencers, etc.). M_sequencer will not exist if we have not registered the sequence with the `uvm_declare_p_sequencer macros.

The drawback of m_sequencer is that once the m_sequencer is defined, one cannot run the sequence on any other sequencer type.

Question 26. What Is The Difference Between Active Mode And Passive Mode With Respect To Agent?

Answer :

An agent is a collection of a sequencer, a driver and a monitor.

In active mode, the sequencer and the driver are constructed and stimulus is generated by sequences sending sequence items to the driver through the sequencer. At the same time the monitor assembles pin level activity into analysis transactions.

In passive mode, only the monitor is constructed and it performs the same function as in an active agent. Therefore, your passive agent has no need for a sequencer. You can set up the monitor using a configuration object.

Question 27. What Is The Difference Between Copy And Clone?

Answer :

The built-in copy () method executes the __m_uvm_field_automation() method with the required copy code as defined by the field macros (if used) and then calls the built-in do copy() virtual function. The built-in do copy () virtual function, as defined in the uvm_object base class, is also an empty method, so if field macros are used to define the fields of the transaction, the built-in copy() method will be populated with the proper code to copy the transaction fields from the field macro definitions and then it will execute the empty do copy() method, which will perform no additional activity.

The copy() method can be used as needed in the UVM test bench. One common place where the copy() method is used is to copy the sampled transaction and pass it into a sb_calc_exp() (scoreboard calculate expected) external function that is frequently used by the scoreboard predictor.

The clone () method calls the create () method (constructs an object of the same type) and then calls the copy() method. It is a one-step command to create and copy an existing object to a new object handle.

Question 28. What Is Uvm Factory?

Answer :

UCM Factory is used to manufacture (create) UVM objects and components. Apart from creating the UVM objects and components the factory concept essentially means that you can modify or substitute the nature of the components created by the factory without making changes to the test bench.

For example, if you have written two driver classes, and the environment uses only one of them. By registering both the drivers with the factory, you can ask the factory to substitute the existing driver in environment with the other type. The code needed to achieve this is minimal, and can be written in the test.

Question 29. What Are The Types Of Sequencer? Explain Each?

Answer :

There are two types of sequencers:

uvm_sequencer #(REQ, RSP) :

When the driver initiates new requests for sequences, the sequencer selects a sequence from a list of available sequences to produce and deliver the next item to execute. In order to do this, this type of sequencer is usually connected to a driver uvm_driver #(REQ, RSP).

uvm_push_sequencer #(REQ, RSP) :

The sequencer pushes new sequence items to the driver, but the driver has the ability to block the item flow when it’s not ready to accept any new transactions. This type of sequencer is connected to a driver of type uvm_push_driver # (REQ, RSP).

30.What is the advantage of `uvm_pre_body and `uvm_post_body ?

uvm sequences has two callback methods pre_body() and post_body(), which are executed before and after the sequence body() method execution. These callbacks are called only when start_sequence() of sequencer or start() method of the sequence is called. User should not call these methods.

```
virtual task pre_body()
virtual task post_body()
```

31.

Types and Enumerations

Field automation

uvm_bitstream_t	The bitstream type is used as a argument type for passing integral values in such methods as uvm_object::set_int_local, uvm_config_int, uvm_printer::print_field, uvm_recorder::record_field, uvm_packer::pack_field and uvm_packer::unpack_field.
uvm_integral_t	The integral type is used as a argument type for passing integral values of 64 bits or less in such methods as uvm_printer::print_field_int, uvm_recorder::record_field_int, uvm_packer::pack_field_int and uvm_packer::unpack_field_int.
uvm_radix_enum	Specifies the radix to print or record in.
uvm_recursion_policy_enum	Specifies the policy for copying objects.
uvm_active_passive_enum	Convenience value to define whether a component, usually an agent, is in "active" mode or "passive" mode.
`uvm_field_* macro flags	Defines what operations a given field should be involved in.

Reporting

uvm_severity	Defines all possible values for report severity.
uvm_action	Defines all possible values for report actions.
uvm_verbosity	Defines standard verbosity levels for reports.

Port Type

uvm_port_type_e	Specifies the type of port

Sequences

uvm_sequencer_arb_mode	Specifies a sequencer's arbitration mode
uvm_sequence_state_enum	Defines current sequence state
uvm_sequence_lib_mode	Specifies the random selection mode of a sequence library

Phasing

uvm_phase_type	This is an attribute of a uvm_phase object which defines the phase type.
uvm_phase_state	The set of possible states of a phase.
uvm_wait_op	Specifies the operand when using methods like uvm_phase::wait_for_state.

Objections

uvm_objection_event	Enumerated the possible objection events one could wait on.

Default Policy Classes

Policy classes copying, comparing, packing, unpacking, and recording uvm_object-based objects.

uvm_default_table_printer	The table printer is a global object that can be used with uvm_object::do_print to get tabular style printing.
uvm_default_tree_printer	The tree printer is a global object that can be used with uvm_object::do_print to get multi-line tree style printing.
uvm_default_line_printer	The line printer is a global object that can be used with uvm_object::do_print to get single-line style printing.
uvm_default_printer	The default printer policy.
uvm_default_packer	The default packer policy.
uvm_default_comparer	The default compare policy.

FIELD AUTOMATION

uvm_bitstream_t

The bitstream type is used as a argument type for passing integral values in such methods
as uvm_object::set_int_local, uvm_config_int, uvm_printer::print_field, uvm_recorder::record_field, uvm_packer::pack_field and uvm_packer::unpack_field.

uvm_integral_t

The integral type is used as a argument type for passing integral values of 64 bits or less in such methods
as uvm_printer::print_field_int, uvm_recorder::record_field_int, uvm_packer::pack_field_int and uvm_packer::unpack_field_int.

uvm_radix_enum

Specifies the radix to print or record in.

UVM_BIN	Selects binary (%b) format
UVM_DEC	Selects decimal (%d) format
UVM_UNSIGNED	Selects unsigned decimal (%u) format
UVM_UNFORMAT2	Selects unformatted 2 value data (%u) format
UVM_UNFORMAT4	Selects unformatted 4 value data (%z) format
UVM_OCT	Selects octal (%o) format
UVM_HEX	Selects hexadecimal (%h) format
UVM_STRING	Selects string (%s) format
UVM_TIME	Selects time (%t) format
UVM_ENUM	Selects enumeration value (name) format
UVM_REAL	Selects real (%g) in exponential or decimal format, whichever format results in the shorter printed output
UVM_REAL_DEC	Selects real (%f) in decimal format
UVM_REAL_EXP	Selects real (%e) in exponential format

uvm_recursion_policy_enum

Specifies the policy for copying objects.

UVM_DEEP	Objects are deep copied (object must implement uvm_object::copy method)
UVM_SHALLOW	Objects are shallow copied using default SV copy.
UVM_REFERENCE	Only object handles are copied.

uvm_active_passive_enum

Convenience value to define whether a component, usually an agent, is in "active" mode or "passive" mode.

UVM_PASSIVE	"Passive" mode
UVM_ACTIVE	"Active" mode

`uvm_field_* macro flags

Defines what operations a given field should be involved in. Bitwise OR all that apply.

UVM_DEFAULT	All field operations turned on
UVM_COPY	Field will participate in uvm_object::copy
UVM_COMPARE	Field will participate in uvm_object::compare
UVM_PRINT	Field will participate in uvm_object::print
UVM_RECORD	Field will participate in uvm_object::record
UVM_PACK	Field will participate in uvm_object::pack
UVM_NOCOPY	Field will not participate in uvm_object::copy
UVM_NOCOMPARE	Field will not participate in uvm_object::compare
UVM_NOPRINT	Field will not participate in uvm_object::print
UVM_NORECORD	Field will not participate in uvm_object::record
UVM_NOPACK	Field will not participate in uvm_object::pack
UVM_DEEP	Object field will be deep copied
UVM_SHALLOW	Object field will be shallow copied
UVM_REFERENCE	Object field will copied by reference
UVM_READONLY	Object field will NOT be automatically configured.

REPORTING

uvm_severity

Defines all possible values for report severity.

UVM_INFO	Informative message.
UVM_WARNING	Indicates a potential problem.
UVM_ERROR	Indicates a real problem. Simulation continues subject to the configured message action.
UVM_FATAL	Indicates a problem from which simulation cannot recover. Simulation exits via $finish after a #0 delay.

uvm_action

Defines all possible values for report actions. Each report is configured to execute one or more actions, determined by the bitwise OR of any or all of the following enumeration constants.

UVM_NO_ACTION	No action is taken
UVM_DISPLAY	Sends the report to the standard output
UVM_LOG	Sends the report to the file(s) for this (severity,id) pair
UVM_COUNT	Counts the number of reports with the COUNT attribute. When this value reaches max_quit_count, the simulation terminates
UVM_EXIT	Terminates the simulation immediately.
UVM_CALL_HOOK	Callback the report hook methods
UVM_STOP	Causes *$stop* to be executed, putting the simulation into interactive mode.
UVM_RM_RECORD	Sends the report to the recorder

uvm_verbosity

Defines standard verbosity levels for reports.

UVM_NONE	Report is always printed. Verbosity level setting cannot disable it.
UVM_LOW	Report is issued if configured verbosity is set to UVM_LOW or above.
UVM_MEDIUM	Report is issued if configured verbosity is set to UVM_MEDIUM or above.
UVM_HIGH	Report is issued if configured verbosity is set to UVM_HIGH or above.
UVM_FULL	Report is issued if configured verbosity is set to UVM_FULL or above.

PORT TYPE

uvm_port_type_e

Specifies the type of port

UVM_PORT	The port requires the interface that is its type parameter.
UVM_EXPORT	The port provides the interface that is its type parameter via a connection to some other export or implementation.
UVM_IMPLEMENTATION	The port provides the interface that is its type parameter, and it is bound to the component that implements the interface.

SEQUENCES

uvm_sequencer_arb_mode

Specifies a sequencer's arbitration mode

UVM_SEQ_ARB_FIFO	Requests are granted in FIFO order (default)

UVM_SEQ_ARB_WEIGHTED	Requests are granted randomly by weight
UVM_SEQ_ARB_RANDOM	Requests are granted randomly
UVM_SEQ_ARB_STRICT_FIFO	Requests at highest priority granted in fifo order
UVM_SEQ_ARB_STRICT_RANDOM	Requests at highest priority granted in randomly
UVM_SEQ_ARB_USER	Arbitration is delegated to the user-defined function, user_priority_arbitration. That function will specify the next sequence to grant.

uvm_sequence_state_enum

Defines current sequence state

UVM_CREATED	The sequence has been allocated.
UVM_PRE_START	The sequence is started and the uvm_sequence_base::pre_start() task is being executed.
UVM_PRE_BODY	The sequence is started and the uvm_sequence_base::pre_body() task is being executed.
UVM_BODY	The sequence is started and the uvm_sequence_base::body() task is being executed.
UVM_ENDED	The sequence has completed the execution of the uvm_sequence_base::body() task.
UVM_POST_BODY	The sequence is started and the uvm_sequence_base::post_body() task is being executed.
UVM_POST_START	The sequence is started and the uvm_sequence_base::post_start() task is being executed.
UVM_STOPPED	The sequence has been forcibly ended by issuing a uvm_sequence_base::kill() on the sequence.
UVM_FINISHED	The sequence is completely finished executing.

uvm_sequence_lib_mode

Specifies the random selection mode of a sequence library

UVM_SEQ_LIB_RAND	Random sequence selection
UVM_SEQ_LIB_RANDC	Random cyclic sequence selection
UVM_SEQ_LIB_ITEM	Emit only items, no sequence execution
UVM_SEQ_LIB_USER	Apply a user-defined random-selection algorithm

PHASING

uvm_phase_type

This is an attribute of a uvm_phase object which defines the phase type.

UVM_PHASE_IMP	The phase object is used to traverse the component hierarchy and call the component phase method as well as the *phase_started* and *phase_ended* callbacks. These nodes are created by the phase macros, `uvm_builtin_task_phase, `uvm_builtin_topdown_phase, and `uvm_builtin_bottomup_phase. These nodes represent the phase type, i.e. uvm_run_phase, uvm_main_phase.
UVM_PHASE_NODE	The object represents a simple node instance in the graph. These nodes will contain a reference to their corresponding IMP object.
UVM_PHASE_SCHEDULE	The object represents a portion of the phasing graph, typically consisting of several NODE types, in series, parallel, or both.
UVM_PHASE_TERMINAL	This internal object serves as the termination NODE for a SCHEDULE phase object.
UVM_PHASE_DOMAIN	This object represents an entire graph segment that executes in parallel with the 'run' phase. Domains may define any network of NODEs and SCHEDULEs. The built-in domain, *uvm*, consists of a single schedule of all the run-time phases, starting with *pre_reset* and ending with *post_shutdown*.

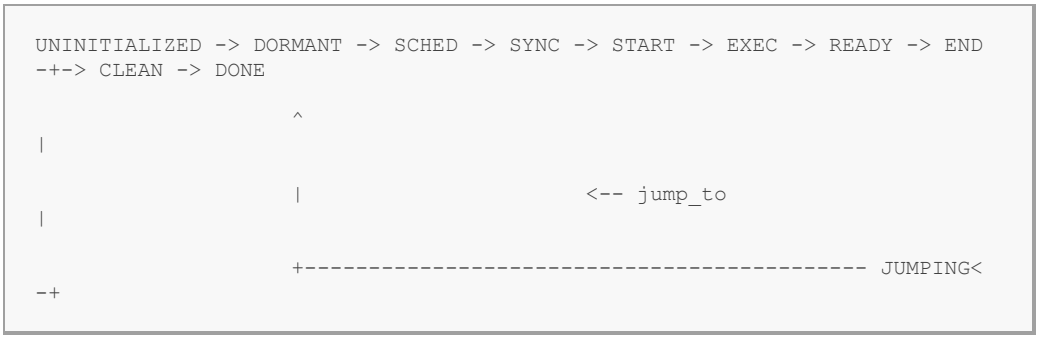
uvm_phase_state

The set of possible states of a phase. This is an attribute of a schedule node in the graph, not of a phase, to maintain independent per-domain state

UVM_PHASE_UNINITIALIZED	The state is uninitialized. This is the default state for phases, and for nodes which have not yet been added to a schedule.
UVM_PHASE_DORMANT	The schedule is not currently operating on the phase node, however it will be scheduled at some point in the future.
UVM_PHASE_SCHEDULED	At least one immediate predecessor has completed. Scheduled phases block until all predecessors complete or until a jump is executed.
UVM_PHASE_SYNCING	All predecessors complete, checking that all synced phases (e.g. across domains) are at or beyond this point
UVM_PHASE_STARTED	phase ready to execute, running phase_started() callback
UVM_PHASE_EXECUTING	An executing phase is one where the phase callbacks are being executed. Its process is tracked by the phaser.
UVM_PHASE_READY_TO_END	no objections remain in this phase or in any predecessors of its successors or in any sync'd phases. This state indicates an opportunity for any phase that needs extra time for a clean exit to raise an objection, thereby causing a return to UVM_PHASE_EXECUTING. If no objection is raised, state will transition to UVM_PHASE_ENDED after a delta cycle. (An example of predecessors of successors: The successor to phase 'run' is 'extract', whose predecessors are 'run' and 'post_shutdown'. Therefore, 'run' will go to this state when both its objections and those of 'post_shutdown' are all dropped.
UVM_PHASE_ENDED	phase completed execution, now running phase_ended() callback

UVM_PHASE_JUMPING	all processes related to phase are being killed and all predecessors are forced into the DONE state.
UVM_PHASE_CLEANUP	all processes related to phase are being killed
UVM_PHASE_DONE	A phase is done after it terminated execution. Becoming done may enable a waiting successor phase to execute.

The state transitions occur as follows



uvm_wait_op

Specifies the operand when using methods like uvm_phase::wait_for_state.

UVM_EQ	equal
UVM_NE	not equal
UVM_LT	less than
UVM_LTE	less than or equal to
UVM_GT	greater than
UVM_GTE	greater than or equal to

OBJECTIONS

uvm_objection_event

Enumerated the possible objection events one could wait on.
See uvm_objection::wait_for.

UVM_RAISED	an objection was raised
UVM_DROPPED	an objection was raised
UVM_ALL_DROPPED	all objections have been dropped

DEFAULT POLICY CLASSES

Policy classes copying, comparing, packing, unpacking, and recording uvm_object-based objects.

uvm_default_table_printer

```
uvm_table_printer uvm_default_table_printer = new()
```

The table printer is a global object that can be used with uvm_object::do_print to get tabular style printing.

uvm_default_tree_printer

```
uvm_tree_printer uvm_default_tree_printer = new()
```

The tree printer is a global object that can be used with uvm_object::do_print to get multi-line tree style printing.

uvm_default_line_printer

```
uvm_line_printer uvm_default_line_printer = new()
```

The line printer is a global object that can be used with uvm_object::do_print to get single-line style printing.

uvm_default_printer

```
uvm_printer uvm_default_printer = uvm_default_table_printer
```

The default printer policy. Used when calls to uvm_object::print or uvm_object::sprintdo not specify a printer policy.

The default printer may be set to any legal uvm_printer derived type, including the global line, tree, and table printers described above.

uvm_default_packer

```
uvm_packer uvm_default_packer = new()
```

The default packer policy. Used when calls to uvm_object::pack and uvm_object::unpack do not specify a packer policy.

uvm_default_comparer

```
uvm_comparer uvm_default_comparer = new()
```

The default compare policy. Used when calls to uvm_object::compare do not specify a comparer policy.

32.What are the types of sequencer? Explain each?

One of the most confusing UVM stuff is about m_sequencer and p_sequencer and the difference between the two. In reality, its just a game of **polymorphism**.

Referring to some forum answer, `m_sequencer` is a **generic sequencer pointer** of type uvm_sequencer_base. It will always exist for a uvm_sequence and is initialized when the sequence is started.

The `p_sequencer` is a **type specific sequencer pointer**, created by registering the sequence to a sequencer using the `` `uvm_declare_p_sequencer `` macros. Being type specific, you will be able to access anything added to the sequencer (i.e. pointers to other sequencers, etc.). `p_sequencer` will not exist if the `` `uvm_declare_p_sequencer `` macros isn't used.

What we are interested in is that `m_sequencer` is a handle of type `uvm_sequencer_base` while `p_sequencer` is a handle of type `user_defined_sequencer`. The `user_defined_sequencer` is a grandchild of `uvm_sequencer_base class`.

When we start a sequence, we provide an object handle of our `user_defined_sequencer`. Internally, in start method, this child class object is assigned into parent handle called `m_sequencer`. So, a **static casting** occurs such that a parent class handle points to child class object (`m_sequencer = user_defined_sequencer_object`).

Now, when referring to sequence, if a `p_sequencer` is defined, the macro `` `uvm_declare_p_sequencer `` expands to a function that declares a `user_defined_sequencer` handle known as `p_sequencer`. This function then casts the `m_sequencer` (parent class handle) back to `p_sequencer` (child class handle) using **dynamic casting** (`$cast`).

32.How set_config_* works?

Configuration is a mechanism in UVM that higher level components in a hierarchy can configure the lower level components variables. Using set_config_* methods, user can configure integer, string and objects of lower level components. Without this mechanism, user should access the lower level component using hierarchy paths, which restricts reusability. This mechanism can be used only with components.
Sequences and transactions cannot be configured using this mechanism. When set_config_* method is called, the data is stored w.r.t strings in a table. There is also a global configuration table.
Higher level component can set the configuration data in level component table. It is the responsibility of the lower level component to get the data from the component table and update the appropriate table.

Set_config_* Methods:

Following are the method to configure integer , string and object of uvm_object based class respectively.

```
function void set_config_int (string inst_name,
string field_name,
uvm_bitstream_t value)

function void set_config_string (string inst_name,
string field_name,
string value)

function void set_config_object (string inst_name,
string field_name,
uvm_object value, bit clone = 1)
```

(S)Arguments description:

- string inst_name: Hierarchical string path.
- string field_name: Name of the field in the table.
- bitstream_t value: In set_config_int, a integral value that can be anything from 1 bit to 4096 bits.
- bit clone : If this bit is set then object is cloned.

inst_name and field_name are strings of hierarchal path. They can include wile card "*" and "?" characters. These methods must be called in build phase of the component.

"*" matches zero or more characters

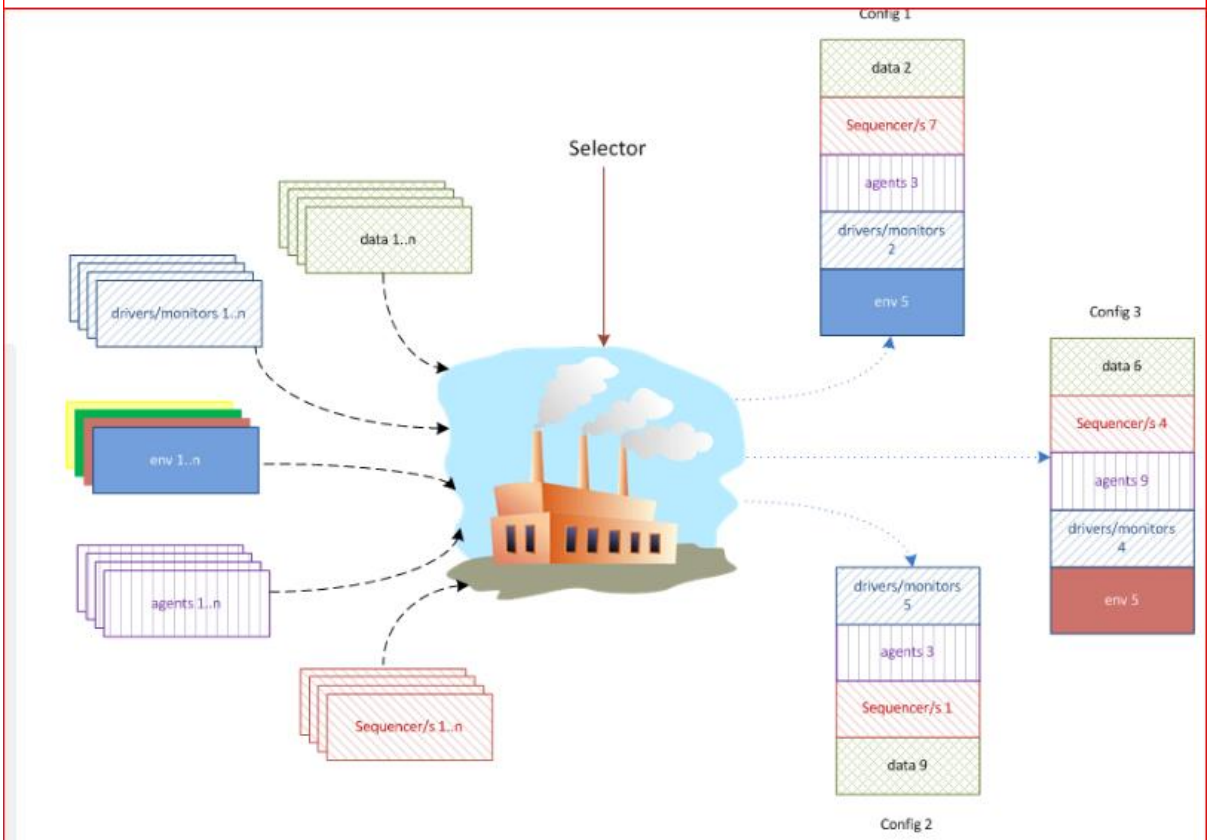
"?" matches exactly one character

33..What is the different between set_config_* and uvm_config_db ?

uvm_config_db is a convenience layer built on top of uvm_resource_db, but that convenience is very important. In particular, uvm_resource_db uses a "last write wins" approach. The uvm_config_db, on the other hand, looks at where things are in the hierarchy up through end_of_elaboration, so "parent wins." Once you start start_of_simulation, the config_db becomes "last write wins."

34. What are the different override types?

To create an instance of a class object, we use `new()` method in SystemVerilog, which can also be used in UVM. But, UVM has introduced the factory concept which essentially means that you can modify or substitute the nature of the components created by the factory without making changes to the testbench. Say, you have written two driver classes, and the environment uses only one of them. By registering both the drivers with the factory, you can ask the factory to substitute the existing driver in environment with the other type. The code needed to achieve this is minimal, and can be written in the test.



A factory can provide an object of the type we specify

So, you have to register all the components that you make with the factory using `uvm_component_utils` or `uvm_object_utils`, and to make it possible to override a component, you have to use the `::type_id::create()` method instead of `new()` whenever you create an object. See the following example.

```
class my_driver extends uvm_driver;
  `uvm_component_utils (my_driver)
  ...
endclass

class my_env extends uvm_env;
  `uvm_component_utils (my_env)

  my_driver drv0;
  ...
  virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);
    drv0 = my_driver::type_id::create ("drv0", this);
  endfunction
endclass
```

Then, you can override these later within the environment/test by calling one of the methods below. Remember that you can print out the configuration using `factory.print()` within the test.

```
// Override all the objects of a particular type

set_type_override_by_type ( uvm_object_wrapper original_type,
                            uvm_object_wrapper override_type,
                            bit replace=1);

set_type_override_by_name ( string original_type_name,
                            string override_type_name,
                            bit replace=1);

// Override a type within a particular instance

set_inst_override_by_type (uvm_object_wrapper original_type,
                           uvm_object_wrapper override_type,
                           string full_inst_path);

set_inst_override_by_name (string original_type_name,
                           string override_type_name,
                           string full_inst_path);
```

1. What is virtual sequence and virtual sequencer/?

A virtual sequence is a sequence which controls stimulus generation using several sequencers. In UVM architecture, sequences, sequencers and drivers are focused on point interfaces and hence there needs to be a higher layer sequence to coordinate stimulus generation across different interfaces and the interactions between them which is what a virtual sequence is.

A virtual sequencer is a uvm_sequencer which contains the handles of all the target sequencers focused on each point interface. A virtual sequence can be started on a virtual sequencer and the virtual sequence can generate and run sub-sequences on target sequencers in a coordinated manner.

Usage of virtual sequencer is a convenient way of handling all target sequencers in a single component and this is useful while creating integrated verification environments (sub-system and system level environments). You can also chain more than one virtual sequencer to a higher layer virtual sequencer in building higher integration testbenches.

--difference between `include and `import in UVM?

'include is a compiler directive and is used for control the execution of compilation of a certain piece of code or module. It basically used for including the contents of one source file into another.You can think of it as similar thing in C which is #include and is a pre-processor directive which is used for including the macro definitions of a source file.

'include is nothing but copying the contents of one file into another.Suppose you have a component class description and you named it as some 123 .sv,now whenever you include this file into another ,for example in 3 separate files like 1 .sv, 2 .sv, 3 .sv, 123 .sv is included as 'include 123 .sv you are actually creating 3 separate classes.

But in case of import it's not the same.Basically there are two types of import one is wildcard (like import xx::*) and the other one is naming (like import xx::a)

Import doesn't do any copying or including the file,it just make the naming visible to that particular class definition.Similar example you can take that of 'include,you can declare a class and in one you include the files and in the rest of two instead of include if you do import then it makes the name visible and each of the two has the same definition.

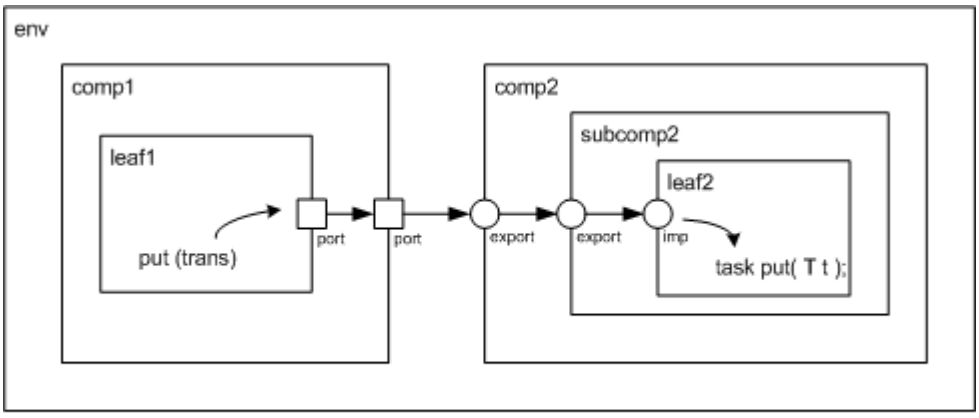
---how to declare multiple imports in uvm?

You don't have to declare anything.

The Import statement is used to make packages available in any place, like this:

Import mypkg ::*;

----What is symbolic representation of port, export and analysis port?



= port = export / imp = analysis port

port-to-port	leaf1's *out* port is connected to its parent's (comp1) *out* port
port-to-export	comp1's *out* port is connected to comp2's *in* export
export-to-export	comp2's *in* export is connected to its child's (subcomp2) *in* export

export-to-imp	subcomp2's *in* export is connected leaf2's *in* imp port.
imp-to-implementation	leaf2's *in* imp port is connected to its implementation, leaf2

----What is the difference in usage of $finish and global stop request in UVM?

There are several mechanism to end the test cases:

a) $finish()

b) global_stop_request()

c) proper usage of objections and it's related callbacks like raised,dropped,all_dropped,etc.

d) set_global_stop_timeout(some value)

e) ovm_test_done/uvm_test_done.

Instead of using global_stop_request if you use $finish to end a test,the problem is that run phase will get interrupted in the middle of run,so it's better to use global stop request.

OVM/UVM test done methods are more used wrt OVM since there is only one run phase and doesn't have any sub division of time taking run phases like uvm. So in case of UVM instead of uvm_test_done other methods mentioned above are beneficial.

---Why we need to register class with uvm factory?

All UVM components, i.e. units which are building your UVM environment should be registered with the factory and also all transient objects. There are 2 exceptions:
(1) there is no need to register configuration objects with the factory.
(2) Components like analysis_ports, exports, tlm_fifo need not be registered with the factory. These are components which are not intended to be replaced.

Factory Method in UVM enables us to register a class, object and variables inside the factory so that we can override their type (if needed) from the test bench without needing to make any significant change in component structure.

--can we use set_config and get_config in sequence ?

set_config_* can be used only for the components not for the sequences.
By using configuration you can change the variables inside components only not in sequences.

But there is a workaround to this problem.

Sequence has handle name called p_sequencer which is pointing the Sequencer on which it is running.
Sequencer is a component , so get_config_* methods are implemented for it.
So from the sequence, using the sequencer get_config_* methods, sequence members can be updated if the variable is configured.

When using set_config_* , path to the variable should be sequencer name, as we are using the sequencer get_config_* method.

---What is uvm_heartbeat ?

Heartbeats provide a way for environments to easily ensure that their descendants are alive. A uvm_heartbeat is associated with a specific objection object. A component that is being tracked by the heartbeat object must raise (or drop) the synchronizing objection during the heartbeat window. The synchronizing objection must be a uvm_callbacks_objection type.

The uvm_heartbeat object has a list of participating objects. The heartbeat can be configured so that all components (UVM_ALL_ACTIVE), exactly one (UVM_ONE_ACTIVE), or any component (UVM_ANY_ACTIVE) must trigger the objection in order to satisfy the heartbeat condition.

uvm_heartbeat

Heartbeats provide a way for environments to easily ensure that their descendants are alive.

METHODS

new	Creates a new heartbeat instance associated with *cntxt*.
set_mode	Sets or retrieves the heartbeat mode.
set_heartbeat	Sets up the heartbeat event and assigns a list of objects to watch.
add	Add a single component to the set of components to be monitored.
remove	Remove a single component to the set of components being monitored.
start	Starts the heartbeat monitor.
stop	Stops the heartbeat monitor.

---how to access DUT signal in uvm_component/uvm_object ?